SOFTWARE ENGINEERING · CHAPTER 01

Lab-01: Multithreading in Java

Five hands-on tasks and one challenge that take you from starting your first thread to building a cancellable, parallel text-processing tool with `java.util.concurrent`.

JDK 25 LTS ≈ 3–4 hours Individual work 5 tasks + 1 challenge

How to work through this lab. Do the tasks in order; each one builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes.

0 Setup — 10 min

1. Install **JDK 25** (Temurin, Oracle or Microsoft build) and verify:

```
java -version
# openjdk version "25" 2025-09-16 LTS
```

2. Create a Maven project (or Gradle with the wrapper) named `lab01-multithreading`. Minimal `pom.xml`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>edu.itc.se</groupId>
  <artifactId>lab01-multithreading</artifactId>
  <version>1.0</version>
  <properties>
    <maven.compiler.release>25</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>6.1.3</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

3. Use one package per task: `edu.itc.se.lab01.task1` … `task5`, `challenge`. Every task has its own `Main` class with a `main` method.
4. Run a class quickly without a build tool during experiments:

```
java src/main/java/edu/itc/se/lab01/task1/Main.java
```

5. Keep `jstack` / `jcmd` handy (they ship with the JDK). You will need them in Task 5.

1 Creating and running threads Easy 25 min

GOAL

Create threads in four different ways, understand the difference between `start()` and `run()`, and coordinate completion with `join()`.

STEPS

1. Write a class `Countdown extends Thread` whose `run()` prints `<name>: 3`, `2`, `1`, `liftoff!`, sleeping 200 ms between lines.
2. Write a class `CountdownTask implements Runnable` that does the same thing, using `Thread.currentThread().getName()`.
3. In `main`, create and start:
   - one `Countdown` named `"apollo"`,
   - one `new Thread(new CountdownTask(), "gemini")`,
   - one thread from a **lambda** named `"mercury"`,
   - one thread built with `Thread.ofVirtual().name("artemis").start(...)`.
4. Print `Mission control: all launched` immediately after starting them, then `join()` all four and print `Mission control: all landed`.
5. Add a flag `--sequential`: when present, call `run()` instead of `start()` on every thread. Observe and explain the difference in a comment at the top of `Main`.
6. Print each thread's `isVirtual()` and `getState()` after the joins.

STARTER CODE

```
package edu.itc.se.lab01.task1;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        boolean sequential = args.length > 0 && args[0].equals("--sequential");
```

```
        Thread apollo  = new Countdown("apollo");
        Thread gemini  = new Thread(new CountdownTask(), "gemini");
        Thread mercury = new Thread(() -> { /* TODO: reuse CountdownTask logic
*/ }, "mercury");
        Thread artemis = Thread.ofVirtual().name("artemis").unstarted(new
CountdownTask());

        // TODO: start (or run) all four depending on 'sequential'
        System.out.println("Mission control: all launched");
        // TODO: join all four
        System.out.println("Mission control: all landed");
        // TODO: print name, isVirtual(), getState() of each
    }
}
```

EXPECTED OUTPUT (CONCURRENT MODE, ORDER WILL VARY)

```
Mission control: all launched
apollo: 3
gemini: 3
mercury: 3
artemis: 3
gemini: 2
apollo: 2
...
artemis: liftoff!
Mission control: all landed
apollo   virtual=false state=TERMINATED
gemini   virtual=false state=TERMINATED
mercury  virtual=false state=TERMINATED
artemis  virtual=true  state=TERMINATED
```

In `--sequential` mode every countdown finishes before the next one starts and "all launched" appears *after* all countdowns, because `run()` executes on the main thread.

Show hints

- You can call `new CountdownTask().run()` from inside the lambda to reuse the logic.
- `Thread.ofVirtual().name("artemis").unstarted(task)` gives you a `Thread` you can start later, just like the others.
- Put the four threads in a `List<Thread>` so you can loop for start and join.
- `Thread.sleep` throws a checked `InterruptedException`; in `run()` catch it and restore the flag with `Thread.currentThread().interrupt()`.

ACCEPTANCE CHECKLIST

- [x] ~~Four threads are created with four different techniques (subclass, Runnable, lambda, virtual builder).~~
- [x] ~~"all launched" prints before the countdowns finish in concurrent mode; "all landed" prints after all of them.~~
- [x] ~~The `--sequential` flag demonstrates `run()` vs `start()`, with a written explanation.~~
- [x] ~~Final states are all `TERMINATED`; only "artemis" reports `virtual=true`.~~

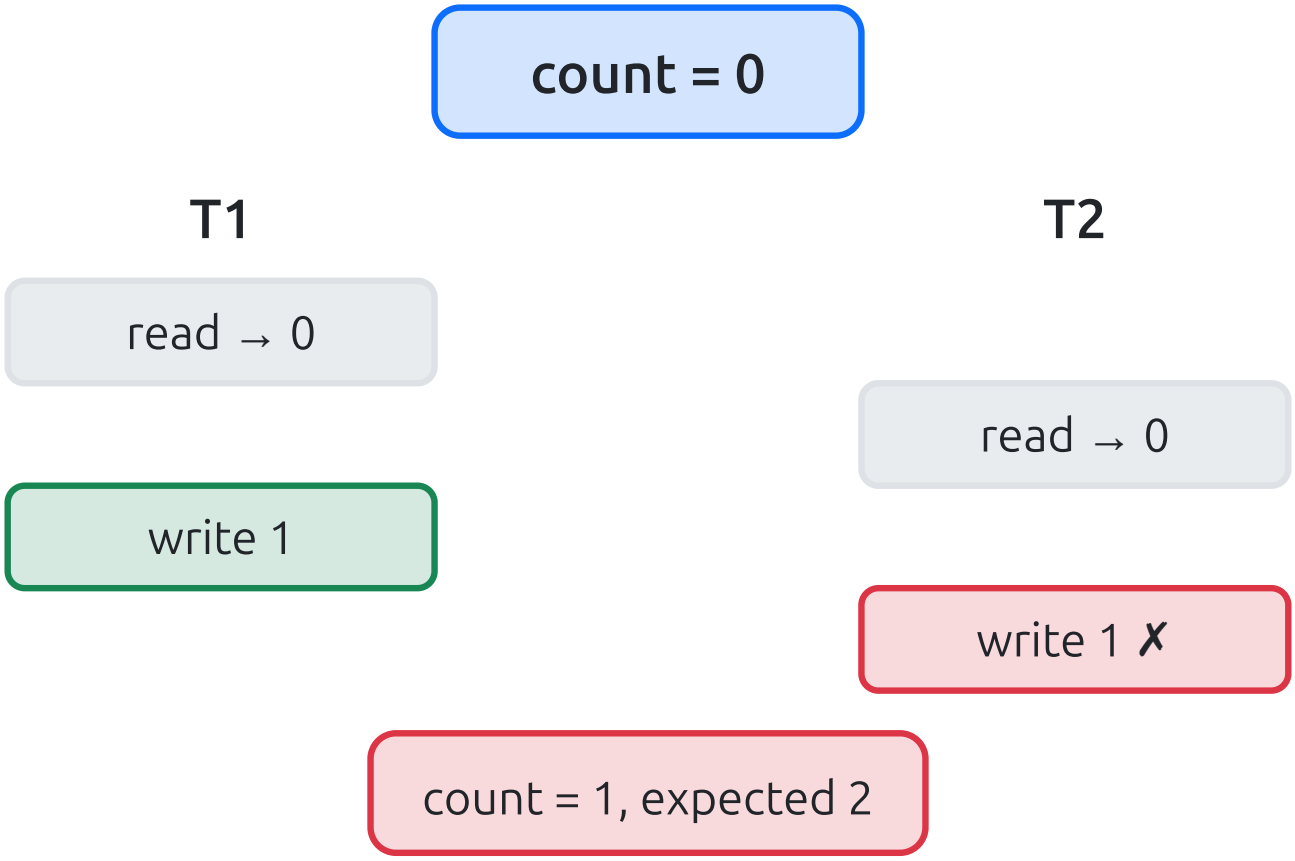
2 Fixing a race condition Easy 30 min

GOAL

Reproduce a lost-update race on a shared counter, then fix it three different ways and compare their performance.

STEPS

1. Define an interface `Counter { void increment(); long get(); }`.
2. Implement `UnsafeCounter` with a plain `long count` field and `count++`.
3. Write a harness `Bench.run(String label, Counter c)` that starts **8 threads**, each incrementing **1 000 000** times, joins them, and prints the label, the final value, whether it equals 8 000 000, and the elapsed milliseconds (`System.nanoTime()`).
4. Run it against `UnsafeCounter` at least three times. Record the wrong results in a comment.
5. Implement and benchmark three correct versions: `SynchronizedCounter` (synchronized methods), `LockCounter` (`ReentrantLock`, unlock in `finally`), `AtomicCounter` (`AtomicLong`).
6. **Bonus:** add `LongAdderCounter` and explain in a comment why it is faster under contention.
7. Answer in `README.md`: why does making only `increment()` synchronized but not `get()` still leave a bug?



STARTER CODE

```
package edu.itc.se.lab01.task2;

import java.util.ArrayList;
import java.util.List;

public class Bench {
    static final int THREADS = 8;
    static final int ITERATIONS = 1_000_000;

    static void run(String label, Counter counter) throws InterruptedException {
        long start = System.nanoTime();
        List<Thread> threads = new ArrayList<>();
        for (int t = 0; t < THREADS; t++) {
            Thread th = new Thread(() -> {
                for (int i = 0; i < ITERATIONS; i++) counter.increment();
            }, "inc-" + t);
            threads.add(th);
            th.start();
        }
        for (Thread th : threads) th.join();
        long ms = (System.nanoTime() - start) / 1_000_000;
        long expected = (long) THREADS * ITERATIONS;
        System.out.printf("%-22s value=%,d  correct=%-5b  %d ms%n",
```

```
                label, counter.get(), counter.get() == expected, ms);
    }

    public static void main(String[] args) throws InterruptedException {
        run("UnsafeCounter", new UnsafeCounter());
        // TODO: SynchronizedCounter, LockCounter, AtomicCounter,
LongAdderCounter
    }
}
```

EXPECTED OUTPUT (NUMBERS WILL DIFFER)

```
UnsafeCounter          value=3,412,907  correct=false  41 ms
SynchronizedCounter    value=8,000,000  correct=true   312 ms
LockCounter            value=8,000,000  correct=true   287 ms
AtomicCounter          value=8,000,000  correct=true   156 ms
LongAdderCounter       value=8,000,000  correct=true   38 ms
```

Show hints

- If `UnsafeCounter` happens to give the right answer, increase `ITERATIONS` or run again; the JIT and the OS scheduler make the race intermittent, which is exactly the point.
- Use `lock.lock(); try { count++; } finally { lock.unlock(); }`. Forgetting `finally` is the classic bug.
- `AtomicLong.incrementAndGet()`; `LongAdder.increment()` then `sum()` to read.
- Unsynchronized `get()` has no happens-before with the writers, so it may return a stale value even after all threads have joined... unless `join()` itself provides the edge. Think about what that means for a reader that does *not* join.

ACCEPTANCE CHECKLIST

- [x] ~~The unsafe version demonstrably produces a wrong total (recorded in a comment).~~
- [x] ~~Three correct implementations all reach exactly 8 000 000 on repeated runs.~~
- [x] ~~The lock is always released in a `finally` block.~~
- [x] ~~Timing results and the `get()` visibility question are discussed in `README.md`.~~

3

Producer–consumer with `wait()`/`notify()` and `BlockingQueue`

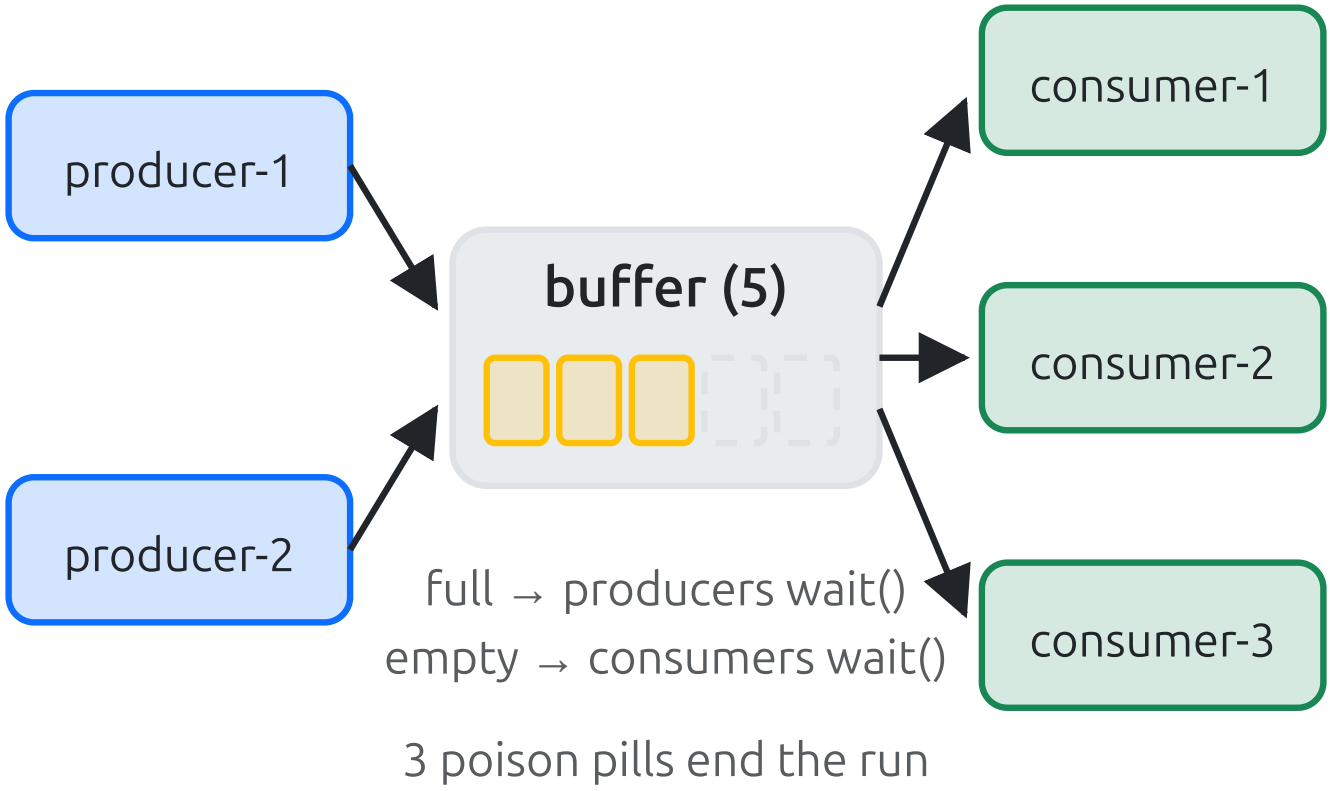
Medium **40 min**

GOAL

Implement a bounded buffer by hand using guarded blocks, then replace it with a `BlockingQueue` and see how much code disappears.

STEPS

1. Create `record Order(int id, String item, int qty)`.
2. Implement `BoundedBuffer<T>` with `put(T)` and `take()` backed by an `ArrayDeque` and a fixed capacity, using `synchronized`, `wait()` in a `while` loop, and `notifyAll()`.
3. Write `Producer implements Runnable`: produces **N orders** (`Order(i, "item-" + i, random 1..5)`) with a random 0–20 ms pause, then puts one **poison pill** per consumer (an `Order` with id −1).
4. Write `Consumer implements Runnable`: takes orders, "processes" them (sleep 5–30 ms), prints `consumer-X processed Order[id=…]`, and stops when it takes a poison pill. Count processed orders in an `AtomicInteger`.
5. In `Main`, run **2 producers × 50 orders** and **3 consumers** with a buffer of capacity **5**. After joining, assert the total processed is 100.
6. Create `Main2` that reuses `Producer`/`Consumer` but uses `ArrayBlockingQueue<Order>(5)` instead of your buffer. Make `Producer`/`Consumer` depend on a small interface so both buffers plug in.
7. Experiment: change `notifyAll()` to `notify()` and the `while` to an `if`. Describe in `README.md` what breaks and why (hint: run it 20 times in a loop).



STARTER CODE

```
package edu.itc.se.lab01.task3;

public interface OrderQueue {
    void put(Order order) throws InterruptedException;
    Order take() throws InterruptedException;
}

// Adapter so a java.util.concurrent queue can be used where OrderQueue is
expected
final class BlockingOrderQueue implements OrderQueue {
    private final java.util.concurrent.BlockingQueue<Order> q;
    BlockingOrderQueue(int capacity) { this.q = new
java.util.concurrent.ArrayBlockingQueue<>(capacity); }
    public void put(Order o) throws InterruptedException { q.put(o); }
    public Order take() throws InterruptedException { return q.take(); }
}

public class BoundedBuffer implements OrderQueue {
    private final java.util.ArrayDeque<Order> items = new java.util.ArrayDeque<>
();
    private final int capacity;
    public BoundedBuffer(int capacity) { this.capacity = capacity; }

    @Override public synchronized void put(Order order) throws
```

```
InterruptedException {
        // TODO: while full → wait(); add; notifyAll()
    }
    @Override public synchronized Order take() throws InterruptedException {
        // TODO: while empty → wait(); remove; notifyAll(); return
        return null;
    }
}
```

EXPECTED OUTPUT (ABRIDGED)

```
consumer-1 processed Order[id=1, item=item-1, qty=3]
consumer-3 processed Order[id=2, item=item-2, qty=1]
consumer-2 processed Order[id=51, item=item-51, qty=5]
...
consumer-2 received poison pill, stopping
consumer-1 received poison pill, stopping
consumer-3 received poison pill, stopping
Total processed: 100 (expected 100) in 1180 ms
```

Show hints

- Each producer must put `consumers / producers` pills, or let `Main` put the pills after joining the producers. The second option is simpler.
- Use `ThreadLocalRandom.current().nextInt(1, 6)` for quantities and pauses.
- With `notify()` + `if` you may wake a producer when a consumer was needed, and a woken thread may find the condition false again: both cause hangs or exceptions from `remove()` on an empty deque.
- Run the whole scenario in a `for (int run = 0; run < 20; run++)` loop to shake out intermittent failures.

ACCEPTANCE CHECKLIST

- [x] ~~`BoundedBuffer` never exceeds its capacity and never returns from an empty buffer (add assertions).~~
- [x] ~~All 100 orders are processed exactly once and every consumer terminates via a poison pill.~~
- [x] ~~The same `Producer`/`Consumer` classes run unchanged against `ArrayBlockingQueue`.~~
- [x] ~~`README.md` explains why `while` + `notifyAll()` is required.~~

4 Thread pools, `Future` and virtual threads **Medium** **40 min**

GOAL

Replace hand-made threads with an `ExecutorService`, collect results through `Future`s, handle failures and timeouts, and compare a platform pool with virtual threads for I/O-bound work.

STEPS

1. **CPU-bound part.** Write `Callable<Long> PrimeCounter(long from, long to)` that counts primes in a range with a naive trial-division test. Count primes below **4 000 000** by splitting the range into **16 chunks** and submitting them to `Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors())`. Sum the `Future` results. Print the total and elapsed time. Compare with a single-threaded run.
2. **I/O-bound part.** Write `Callable<String> FakeDownload(String url)` that sleeps 200 ms (simulated network) and returns `url + " -> " + length`. Submit **500** downloads to (a) a fixed pool of 20 platform threads and (b) `Executors.newVirtualThreadPerTaskExecutor()`. Print elapsed time for each. Explain the difference in `README.md`.
3. **Failure handling.** Make every 50th download throw `IOException("boom")`. Collect results with `future.get()`, catch `ExecutionException`, and print a summary `ok=490 failed=10`. The program must not crash.
4. **Timeouts.** Make one download sleep 5 s. Use `future.get(1, TimeUnit.SECONDS)`, catch `TimeoutException`, call `future.cancel(true)`, and report it as `timed out`.
5. **Shutdown.** Use try-with-resources for the executors (JDK 19+), or `shutdown()` + `awaitTermination()` + `shutdownNow()`. Verify the JVM exits without `System.exit`.
6. **Bonus.** Rewrite the I/O part with `CompletableFuture.supplyAsync(..., executor)` and `allOf`, using `handle()` to turn failures into a fallback string.

STARTER CODE

```
package edu.itc.se.lab01.task4;

import java.util.*;
import java.util.concurrent.*;

public class Main {
    static long countPrimes(long from, long to) {
        long count = 0;
        for (long n = Math.max(from, 2); n < to; n++) {
            boolean prime = true;
            for (long d = 2; d * d <= n; d++) if (n % d == 0) { prime = false;
break; }
            if (prime) count++;
        }
```

```
        return count;
    }

    public static void main(String[] args) throws Exception {
        final long LIMIT = 4_000_000, CHUNKS = 16;

        long t0 = System.nanoTime();
        long single = countPrimes(0, LIMIT);
        System.out.printf("single-threaded: %d primes in %d ms%n", single,
(System.nanoTime() - t0) / 1_000_000);

        t0 = System.nanoTime();
        try (ExecutorService pool =
Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors())) {
            List<Future<Long>> futures = new ArrayList<>();
            // TODO: submit CHUNKS PrimeCounter callables covering [0, LIMIT)
            long total = 0;
            // TODO: sum futures
            System.out.printf("pool: %d primes in %d ms%n", total,
(System.nanoTime() - t0) / 1_000_000);
        }

        // TODO: I/O-bound part with 500 FakeDownload tasks on platform pool vs
virtual threads
        // TODO: failures every 50th task, one 5-second task with a 1-second
timeout
    }
}
```

EXPECTED OUTPUT (TIMINGS DEPEND ON YOUR MACHINE)

```
single-threaded: 283146 primes in 2210 ms
pool (8 threads): 283146 primes in 420 ms
downloads on fixed(20) platform threads : ok=489 failed=10 timedOut=1 in 5210 ms
downloads on virtual threads            : ok=489 failed=10 timedOut=1 in 1230 ms
```

Show hints

- Chunk boundaries: `from = i * LIMIT / CHUNKS`, `to = (i + 1) * LIMIT / CHUNKS`. Chunks near the top are slower; that is fine.
- `ExecutionException.getCause()` holds the original `IOException`.
- 500 tasks × 200 ms on 20 threads ≈ 5 s; on virtual threads all 500 sleep concurrently ≈ 200 ms plus overhead. The slow 5-second task dominates unless you cancel it.
- Make the timed-out task the *first* you wait on so the 1-second timeout is measurable; alternatively wait on the others first and note how `get()` then returns immediately.

- If the JVM does not exit, a non-daemon pool thread is still alive: you forgot to shut a pool down.

ACCEPTANCE CHECKLIST

- [x] ~~Parallel prime count equals the single-threaded count and is measurably faster on a multi-core machine.~~
- [x] ~~Virtual-thread run of the I/O tasks is several times faster than the 20-thread pool, with an explanation.~~
- [x] ~~Failed tasks and the timed-out task are reported in a summary; the program never crashes with a stack trace.~~
- [x] ~~All executors are closed or shut down; the JVM exits by itself.~~

5 Diagnosing a deadlock and shutting down gracefully Hard

45 min

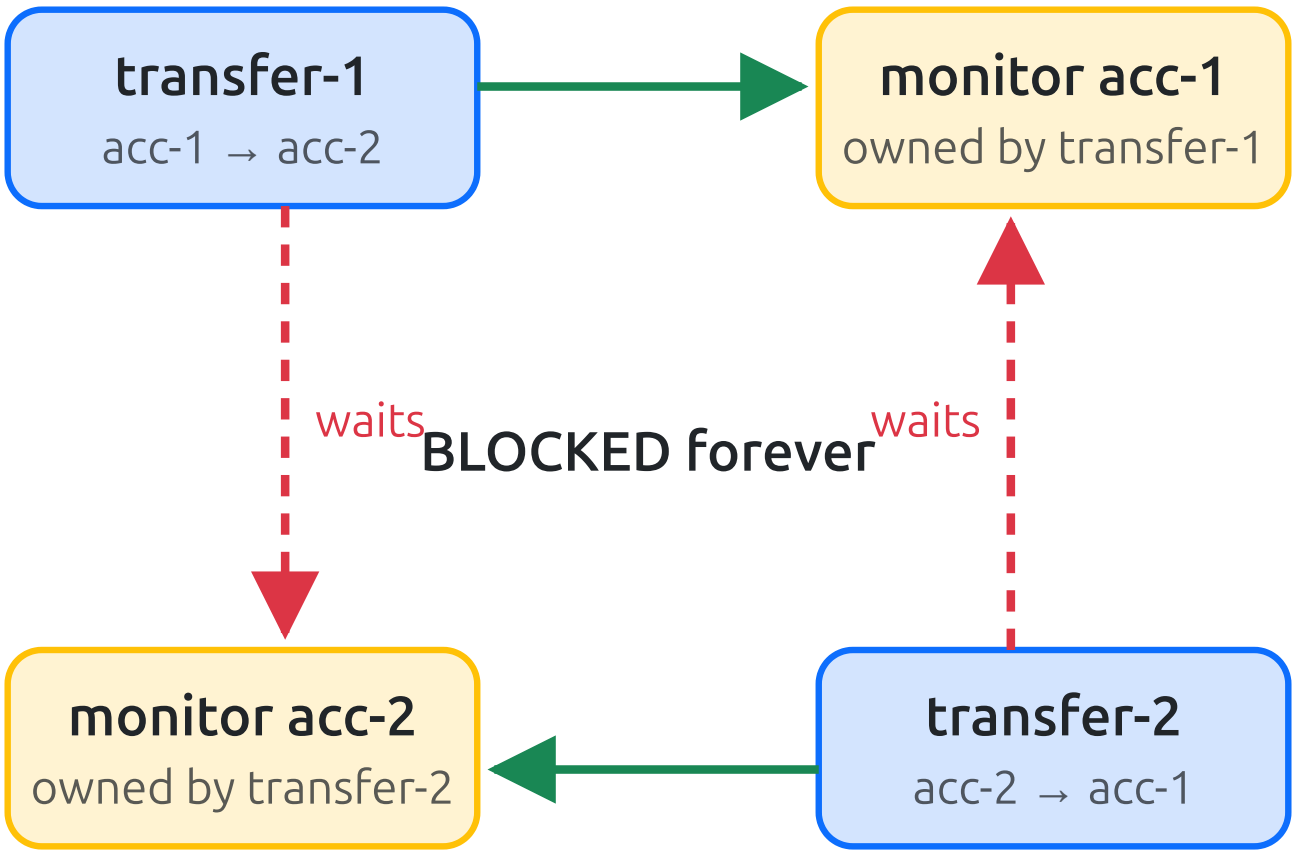
GOAL

Reproduce a real deadlock, find it with JDK tools, fix it with lock ordering and with `tryLock`, and add cooperative cancellation via interruption.

STEPS

1. Copy the deadlocking `Bank` below. Run it; it should hang within a second. If it doesn't, increase the number of transfer threads.
2. While it hangs, find the process id with `jcmd` (or `jps`) and run `jstack <pid>`. Save the section starting with `Found one Java-level deadlock` to `deadlock.txt` and include it in your submission.
3. Add a **watchdog** daemon thread that every second calls `ManagementFactory.getThreadMXBean().findDeadlockedThreads()` and prints the names of deadlocked threads.
4. **Fix A**: `OrderedBank` acquires the two account monitors in ascending `id` order. Prove with 8 threads × 10 000 random transfers that the total balance stays constant and nothing hangs.
5. **Fix B**: `TryLockBank` gives each account a `ReentrantLock` and uses `tryLock(10, MILLISECONDS)` on both, releasing and retrying with a random back-off on failure. Count the retries.

6. **Graceful shutdown**: make the transfer workers loop `while (!Thread.currentThread().isInterrupted())`. After 2 seconds, `main` interrupts all workers, joins them with a 1-second timeout, and prints how many transfers completed and the final total balance. Workers must restore the interrupt flag when a blocking call throws `InterruptedException`.



STARTER CODE (DELIBERATELY BROKEN)

```
package edu.itc.se.lab01.task5;

import java.util.concurrent.ThreadLocalRandom;

public class Bank {
    static final class Account {
        final int id; private long balance;
        Account(int id, long balance) { this.id = id; this.balance = balance; }
        long balance() { return balance; }
        void withdraw(long a) { balance -= a; }
        void deposit(long a)  { balance += a; }
    }

    // BUG: lock order depends on the argument order → circular wait
```

```
    static void transfer(Account from, Account to, long amount) {
        synchronized (from) {
            synchronized (to) {
                if (from.balance() >= amount) { from.withdraw(amount);
to.deposit(amount); }
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Account[] accounts = new Account[4];
        for (int i = 0; i < accounts.length; i++) accounts[i] = new Account(i,
1_000);

        Thread[] workers = new Thread[8];
        for (int w = 0; w < workers.length; w++) {
            workers[w] = new Thread(() -> {
                var rnd = ThreadLocalRandom.current();
                for (int i = 0; i < 100_000; i++) {
                    Account a = accounts[rnd.nextInt(accounts.length)];
                    Account b = accounts[rnd.nextInt(accounts.length)];
                    if (a != b) transfer(a, b, rnd.nextInt(1, 50));
                }
            }, "transfer-" + w);
            workers[w].start();
        }
        for (Thread t : workers) t.join();
        long total = 0; for (Account a : accounts) total += a.balance();
        System.out.println("total = " + total + " (expected 4000)");   // never
printed – deadlock
    }
}
```

EXPECTED OUTPUT

```
# broken version, with watchdog
[watchdog] DEADLOCK detected between: transfer-3, transfer-6
(process hangs – Ctrl+C)

# jstack excerpt to include in deadlock.txt
Found one Java-level deadlock:
=============================
"transfer-3":
  waiting to lock monitor 0x... (object 0x..., a Bank$Account),
  which is held by "transfer-6"
"transfer-6":
  waiting to lock monitor 0x... (object 0x..., a Bank$Account),
```

```
    which is held by "transfer-3"

# OrderedBank
transfers=800000 total=4000 (expected 4000) in 640 ms

# TryLockBank
transfers=800000 retries=1327 total=4000 (expected 4000) in 910 ms

# graceful shutdown
[main] interrupting 8 workers after 2000 ms
transfer-0 stopped after 118203 transfers
...
all workers stopped=true total=4000 (expected 4000)
```

Show hints

- Lock ordering: `Account first = a.id < b.id ? a : b; Account second = a.id < b.id ? b : a;` and always `synchronized (first) { synchronized (second) { … } }`.
- `ThreadMXBean.findDeadlockedThreads()` returns thread ids or `null`; map them to names with `getThreadInfo(ids)`.
- In `TryLockBank`, on failure to get the second lock you must release the first before backing off; otherwise you have re-created hold-and-wait.
- Interruption: a worker doing pure CPU work never throws, so the `while (!isInterrupted())` check is what stops it. Only the back-off `sleep` in `TryLockBank` will throw.
- To make `join(1000)` meaningful, print `t.isAlive()` after it for every worker.

ACCEPTANCE CHECKLIST

- [x] ~~`deadlock.txt` contains a genuine `jstack` deadlock report from your machine.~~
- [x] ~~The watchdog detects and names the deadlocked threads at runtime.~~
- [x] ~~Both fixes complete 800 000 transfers with the total balance unchanged; the tryLock version reports its retry count.~~
- [x] ~~Workers stop within 1 s of `interrupt()`, restore the interrupt flag, and the invariant still holds after shutdown.~~

★ Challenge: parallel word-frequency analyser Hard

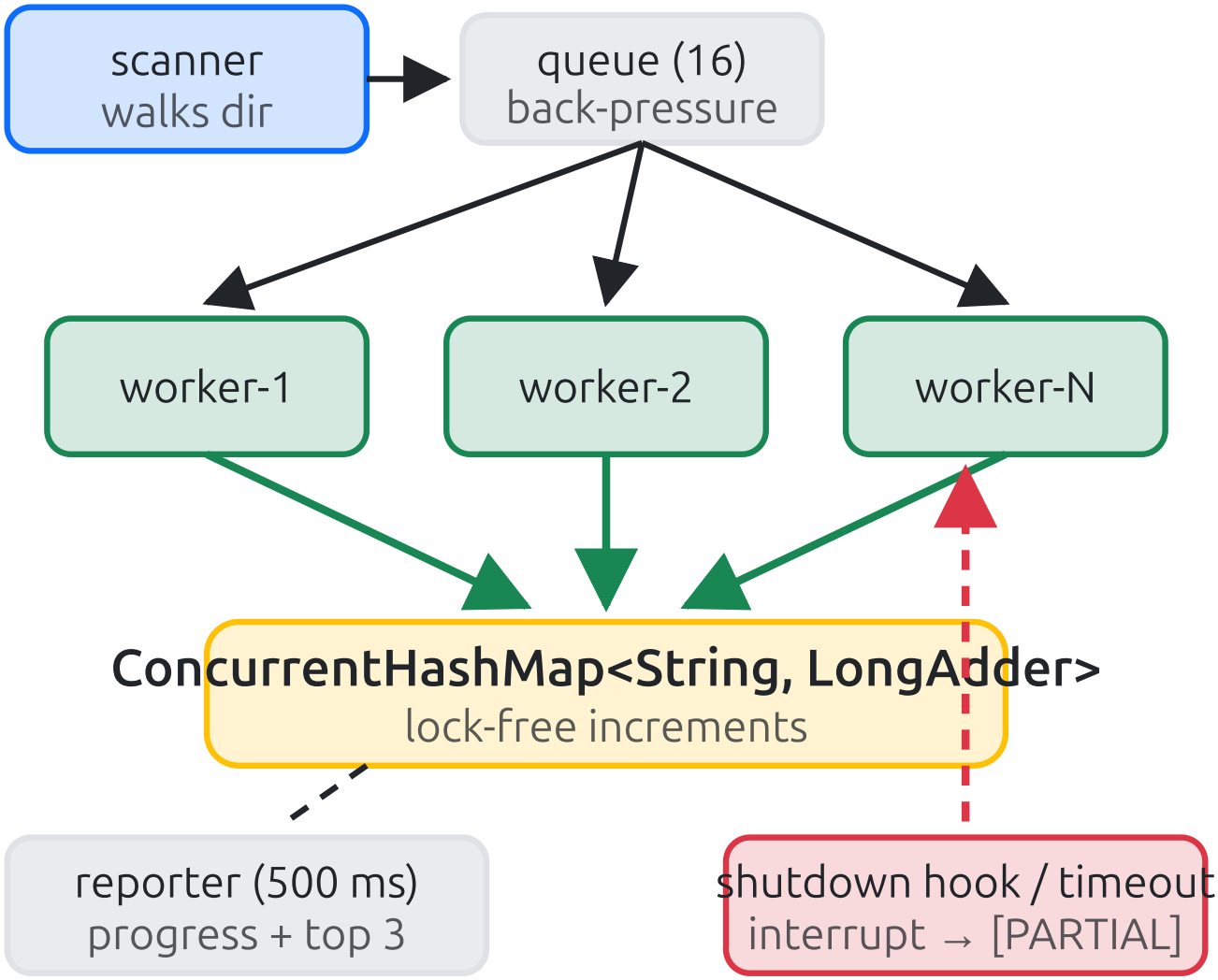
90 min · optional, extra credit

SCENARIO

Your team needs a command-line tool that scans a directory of text files (think: server logs or e-books) and reports the most frequent words. The files are large and numerous, so the tool must process them **in parallel**, show **live progress**, and **stop cleanly** when the user presses Ctrl+C or a deadline passes. It must use only the JDK.

REQUIREMENTS

1. **Input.** `java … WordCount <dir> [--top N] [--timeout SECONDS] [--virtual]`. Generate test data with a small helper that writes 200 files of ~50 000 random words each (you may reuse a fixed vocabulary of 2 000 words).
2. **Pipeline.** A single *scanner* thread walks the directory and puts file paths into a bounded `BlockingQueue<Path>` (capacity 16) so the scanner cannot get far ahead (back-pressure). *N worker* threads (default = cores, or virtual threads with `--virtual`) take paths, read the file line by line, split on non-letters, lower-case, and update a shared `ConcurrentHashMap<String, LongAdder>` (or `merge` with `Long`). Use a poison pill or a `CountDownLatch` to signal completion.
3. **Progress.** A *reporter* thread prints every 500 ms: files done / total, words counted, throughput in words/s, and the current top 3 words. It must not block the workers (read counters atomically, never lock the whole map).
4. **Cancellation.** A `Runtime.getRuntime().addShutdownHook(...)` and the `--timeout` deadline both trigger the same `shutdown()` method: interrupt the scanner and workers, wait at most 2 s, then print a **partial** result clearly marked `[PARTIAL]`. Workers must check the interrupt flag between files and restore it if `take()` throws.
5. **Result.** Print the top N words with counts and percentage, total words, total files, elapsed time, and the worker mode used. Also write `result.csv`.
6. **Verification.** Provide a single-threaded reference implementation and a JUnit 6 test asserting that both produce identical maps for a 20-file fixture. Add a test that the tool stops within 3 s when given `--timeout 1` on a directory of 5 000 files.
7. **Report.** In `README.md`: a table of elapsed time for 1, 2, 4, 8 platform workers and virtual threads; a short explanation of why `LongAdder`/`merge` beats `synchronized` on the map; and one paragraph on which of the Coffman conditions your design rules out and how.



SKELETON

```
package edu.itc.se.lab01.challenge;

import java.nio.file.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class WordCount {
    private final BlockingQueue<Path> queue = new ArrayBlockingQueue<>(16);
    private final ConcurrentHashMap<String, LongAdder> counts = new
ConcurrentHashMap<>();
    private final AtomicInteger filesDone = new AtomicInteger();
    private final LongAdder wordsDone = new LongAdder();
    private final AtomicBoolean stopping = new AtomicBoolean();
    private static final Path POISON = Path.of("__POISON__");

    private final Path dir; private final int workers; private final boolean
virtual;
    private final int top; private final long timeoutSec;
```

```
    WordCount(Path dir, int workers, boolean virtual, int top, long timeoutSec)
{ /* … */ }

    void run() throws InterruptedException {
        ExecutorService pool = virtual ?
Executors.newVirtualThreadPerTaskExecutor()
                                       : Executors.newFixedThreadPool(workers);
        Thread scanner = Thread.ofPlatform().name("scanner").start(this::scan);
        ScheduledExecutorService reporter =
Executors.newSingleThreadScheduledExecutor();
        reporter.scheduleAtFixedRate(this::report, 500, 500,
TimeUnit.MILLISECONDS);
        Runtime.getRuntime().addShutdownHook(new Thread(this::shutdown,
"shutdown-hook"));

        // TODO: submit 'workers' instances of this::work, arrange completion
(latch / poison),
        //       enforce --timeout, then printResult(partial?) and stop
everything.
    }

    private void scan() { /* Files.walk(dir) → queue.put(path); finally put one
POISON per worker */ }

    private void work() {
        try {
            while (!Thread.currentThread().isInterrupted()) {
                Path p = queue.take();
                if (p == POISON) return;
                countFile(p);
                filesDone.incrementAndGet();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void countFile(Path p) { /* read lines, split on [^\p{L}]+, lower-
case, counts.computeIfAbsent(w, k -> new LongAdder()).increment() */ }

    private void report() { /* files done / total, words, words/s, top 3 – never
lock the map */ }

    void shutdown() { if (stopping.compareAndSet(false, true)) { /* interrupt
scanner + workers, await ≤ 2 s, printResult(true) */ } }

    void printResult(boolean partial) { /* sort entries by count desc, print top
```

```
N with %, write result.csv */ }

    public static void main(String[] args) throws Exception { /* parse args, new
WordCount(...).run() */ }
}
```

EXPECTED OUTPUT

```
$ java -cp target/classes edu.itc.se.lab01.challenge.WordCount ./data --top 5 --
timeout 30
[0.5s] files 37/200  words 1,850,000  3,700,000 w/s  top: the(92,110)
and(61,003) of(58,921)
[1.0s] files 79/200  words 3,950,000  3,950,000 w/s  top: the(196,540)
and(130,112) of(125,880)
[1.5s] files 121/200 words 6,050,000  4,033,000 w/s  top: the(301,220)
and(199,431) of(192,015)
[2.0s] files 163/200 words 8,150,000  4,075,000 w/s  top: the(405,900)
and(268,720) of(258,650)
DONE  files=200 words=10,000,000 elapsed=2.46s mode=platform(8)
 rank  word        count     share
    1  the       497,812    4.98 %
    2  and       329,540    3.30 %
    3  of        317,220    3.17 %
    4  to        301,004    3.01 %
    5  a         280,117    2.80 %
result.csv written

$ java … WordCount ./data --top 3 --timeout 1
[0.5s] files 37/200 ...
[1.0s] deadline reached – stopping
[PARTIAL] files=81 words=4,050,000 elapsed=1.12s mode=platform(8)
 ...
```

Show hints

- Read files with `Files.lines(p)` in a try-with-resources; split with a precompiled `Pattern.compile("[^\\p{L}]+")`. Compiling the pattern per line is a common hot spot.
- `counts.computeIfAbsent(word, k -> new LongAdder()).increment()` is atomic and lock-free on the hot path; compare with `counts.merge(word, 1L, Long::sum)` in your README.
- The reporter can snapshot the top 3 by streaming `counts.entrySet()`; it's a weakly consistent view, which is acceptable for progress output. Say so in a comment.
- For the timeout, `latch.await(timeoutSec, TimeUnit.SECONDS)` returning `false` is your deadline signal. Use the same `shutdown()` for the hook; the `AtomicBoolean` guarantees it runs once.
- Shutdown hooks run while the JVM is exiting, so do not call `System.exit` inside one, and keep it under a couple of seconds.

- To make the scanner interruptible while blocked on a full queue, `put()` already throws `InterruptedException`: catch it, restore the flag, and exit the walk.

ACCEPTANCE CHECKLIST

- [x] ~~Parallel and single-threaded results are identical (JUnit test passes).~~
- [x] ~~Progress lines appear every 500 ms without slowing workers; the bounded queue limits memory.~~
- [x] ~~Ctrl+C and `--timeout` both produce a `[PARTIAL]` result within 3 s and the JVM exits without stray threads.~~
- [x] ~~`README.md` contains the scaling table, the LongAdder/merge explanation, and the Coffman-condition paragraph.~~

Grading rubric

Component	Points	Full marks when…
Task 1 · Creating & running threads	10	Four techniques, correct join, run vs start explained.
Task 2 · Race condition	15	Race reproduced, three correct fixes, timings and visibility question discussed.
Task 3 · Producer–consumer	20	Correct guarded blocks, poison pills, pluggable BlockingQueue, notify/if failure explained.
Task 4 · Executors & futures	20	Parallel speed-up, virtual vs platform comparison, failures and timeouts handled, clean shutdown.
Task 5 · Deadlock & interruption	25	jstack evidence, watchdog, both fixes verified, graceful interruption.
Code quality	10	Named threads, no swallowed InterruptedException, locks released in finally, README complete.
★ Challenge (extra credit)	+20	All acceptance items ticked; tests pass; report includes scaling table.
Total	**100 (+20)**	

Automatic deductions: −5 for each empty `catch (InterruptedException e) {}`; −5 for any use of `Thread.stop()`; −10 if the program relies on `Thread.sleep` for ordering instead of `join`/latches/futures; −10 if any executor is never shut down.

Submission

1. Repository layout:

```
lab01-multithreading/
├── pom.xml
├── README.md            ← answers, timings, explanations for every task
├── deadlock.txt         ← jstack output from Task 5
└── src/
    ├── main/java/edu/itc/se/lab01/
    │   ├── task1/ … task5/
    │   └── challenge/
    └── test/java/edu/itc/se/lab01/challenge/WordCountTest.java
```

2. Make sure `mvn -q verify` (or `./gradlew build`) succeeds from a clean checkout on JDK 25.
3. Push to your course Git repository on a branch named `lab-01` and open a merge request titled `Lab-01 – <your name>`.
4. Include the screenshot or terminal capture of the deadlock detection and of the `[PARTIAL]` result if you attempted the challenge.

Back to Chapter 01 slides | Retake the chapter quiz